HACKATHON ASSESSMENT

5G NETWORK FAULT DETECTION AND RECOVERY SYSTEM

A Multithreaded C-Based Tower Monitoring and Recovery Solution

Course Name: C Programming and Systems Programming

Course Code: CSA0203

Presented by:
S.S.Sai Chandan(192312190)
M.Charan Teja Reddy(192312232)
Obulesh(192312243)

1. Introduction

Fifth-generation (5G) mobile networks consist of a very large number of radio access points, small cells and macro towers that must operate continuously to provide reliable high-speed connectivity. Each tower generates operational information such as signal strength, available bandwidth, number of connected users, equipment temperature and fault status. In a large telecom deployment, these measurements arrive continuously and may be updated by several monitoring processes at the same time. A fault management system therefore needs to be fast, concurrent and reliable rather than depending on a single sequential monitoring loop.

This hackathon project develops a 5G Network Fault Detection and Recovery System in C. The program models multiple towers as dynamically allocated records and uses pointers to update individual records efficiently. POSIX threads process independent network streams concurrently, while mutexes protect shared tower data and semaphores coordinate alert/recovery work. A TCP socket interface represents communication between towers and a central control center. Historical fault events are stored in a log file so that operators can inspect previous failures. Inter-process communication (IPC) is used to pass a recovery request from the fault detection process to the recovery process.

The key hackathon requirement is non-blocking fault handling: when one tower fails, the system should identify the failure, generate an alert, record the event and initiate recovery without stopping the processing of healthy towers. This is achieved by separating monitoring, detection and recovery activities and by protecting only the critical shared-data sections with synchronization primitives.

2. System Objectives

- Maintain dynamically allocated records for thousands of 5G towers.
- Use pointers and structures for efficient real-time updates.
- Detect low signal, insufficient bandwidth, overheating and explicit fault conditions.
- Process multiple tower streams concurrently using pthreads.
- Prevent race conditions with mutexes and coordinate work with semaphores.
- Send tower information through a TCP socket to a control-center service.
- Create an historical fault log for auditing and troubleshooting.
- Use a pipe-based IPC mechanism between fault detection and recovery processes.
- Continue monitoring healthy towers while a failed tower is being recovered.

3. Proposed System Architecture

The proposed architecture contains five logical layers. Tower data is represented by a dynamically allocated array of structures. Stream-processing threads update the records. A fault detector evaluates the current values against threshold conditions. When a fault is detected, the event is logged and a recovery message is sent through IPC. A recovery worker then changes the tower state back to operational after simulating a repair action.

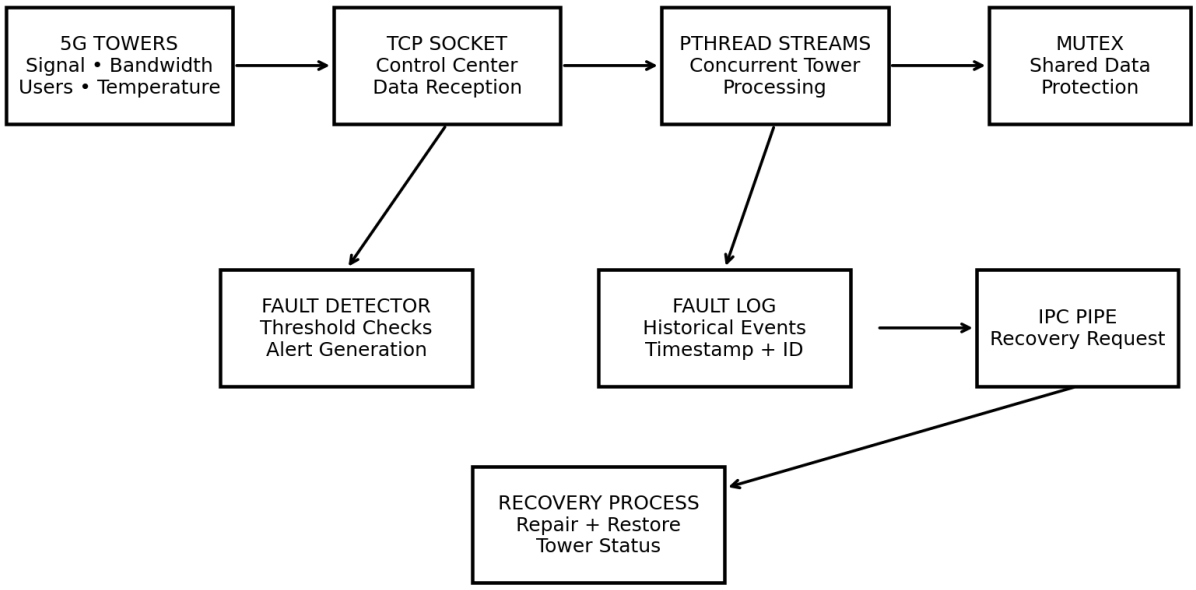

Figure 1. High-level architecture of the 5G fault detection and recovery system.

4. Major Design Components

Dynamic tower database: A Tower structure stores ID, signal strength, bandwidth, users, temperature and fault status. Memory is allocated dynamically with calloc/realloc so the design can scale.

Concurrent stream processing: Each monitoring stream is handled by a pthread. Threads update tower measurements without requiring the entire system to stop.

Synchronization: A pthread mutex protects the shared tower array and log file. A POSIX semaphore demonstrates event synchronization for fault processing.

Fault detection: The detector flags explicit faults and abnormal measurements, such as signal below -90 dBm, bandwidth below 50 Mbps or temperature above 75 °C.

Recovery: A separate recovery process receives a tower ID over a pipe, simulates corrective action and clears the fault state.

Socket communication: A TCP server socket provides a control-center endpoint. A tower/client can send a formatted telemetry message to the server.

Historical logging: Each detected fault is appended to fault_history.log with the tower ID, reason and recovery action.

5. Fault Detection and Recovery Workflow

1. Allocate the tower database dynamically and initialize tower records.

2. Start the control-center socket server and create monitoring threads.

3. Each thread receives or simulates telemetry for its assigned tower.

4. Lock the shared record with a mutex and update the tower using a pointer.

5. Run fault-detection checks after the update.

6. If a fault is present, generate an alert and append the event to the historical log.

7. Send the affected tower ID through the IPC pipe to the recovery process.

8. The recovery process repairs the tower while other pthreads continue monitoring healthy towers.

9. Update the tower state to recovered and continue the monitoring cycle.

6. Hackathon Challenge Handling

The central challenge is avoiding a system-wide pause when a tower fails. A naïve implementation might perform fault detection and recovery inside the same monitoring loop, causing all other towers to wait. In the proposed design, monitoring is concurrent and the recovery operation is separated through IPC. The mutex is held only while a shared record is being modified or read, so the simulated recovery delay does not block unrelated towers. Consequently, a failed tower can enter recovery while healthy towers continue producing telemetry and status output.

7. Expected Advantages

- Scalable memory organization for large tower populations.
- Improved responsiveness through parallel stream processing.
- Safe access to shared data through synchronization.
- Persistent fault history for maintenance analysis.
- Process isolation between fault detection and recovery.
- Network-ready design using TCP sockets.
- Non-blocking recovery behavior suitable for large 5G deployments.

8. C Program Implementation

The following program is a compact demonstration of the complete design. It includes dynamic allocation, pointers, pthread-based stream processing, mutex/semaphore synchronization, file logging, command-line configuration, TCP socket communication, and pipe-based IPC. Compile on Linux with pthread support.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <sys/wait.h>
#include <time.h>


#define MAX_TOWERS 8
#define STREAMS 4


typedef struct {
    int id;
    float signal;
    float bandwidth;
    int users;
    float temperature;
    int fault;
} Tower;


Tower *towers;
pthread_mutex_t data_lock = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t log_lock = PTHREAD_MUTEX_INITIALIZER;
sem_t fault_sem;
int recovery_fd;


void log_fault(Tower *t, const char *reason) {
    FILE *fp;
    pthread_mutex_lock(&log_lock);
    fp = fopen("fault_history.log", "a");
    if (fp) {
        time_t now = time(NULL);
        fprintf(fp, "%sTower %d: %s -> recovery requested\n",
                ctime(&now), t->id, reason);
        fclose(fp);
    }
    pthread_mutex_unlock(&log_lock);
}


void *monitor_stream(void *arg) {
    int stream = *(int *)arg;
    for (int k = 0; k < 4; k++) {
        int id = (stream * 2 + k) % MAX_TOWERS;
        Tower *t = &towers[id];
```

```
        pthread_mutex_lock(&data_lock);
        t->signal = -70.0f - (float)((stream + k) * 6);
        t->bandwidth = 120.0f - (float)(k * 25);
        t->users = 40 + stream * 10 + k * 4;
        t->temperature = 55.0f + (float)(k * 8);
        t->fault = (t->signal < -90 || t->bandwidth < 50 ||
                    t->temperature > 75);
        printf("[STREAM %d] Tower %d  Signal %.1f dBm  BW %.1f Mbps  "
               "Temp %.1f C  Status: %s\n",
               stream, t->id, t->signal, t->bandwidth,
               t->temperature, t->fault ? "FAULT" : "HEALTHY");


        if (t->fault) {
            log_fault(t, "Abnormal telemetry");
            write(recovery_fd, &t->id, sizeof(t->id));
            sem_post(&fault_sem);
        }
        pthread_mutex_unlock(&data_lock);
        usleep(180000);
    }
    return NULL;
}


void recovery_process() {
    int id;
    while (read(STDIN_FILENO, &id, sizeof(id)) > 0) {
        sem_wait(&fault_sem);
        printf("[RECOVERY] Tower %d recovery initiated\n", id);
        usleep(250000);
        printf("[RECOVERY] Tower %d restored and returned to service\n", id);
    }
    exit(0);
}


int start_server(int port) {
    int s = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in a = {0};
    a.sin_family = AF_INET;
    a.sin_addr.s_addr = htonl(INADDR_ANY);
    a.sin_port = htons(port);
    bind(s, (struct sockaddr *)&a, sizeof(a));
    listen(s, 5);
    printf("[CONTROL CENTER] TCP server ready on port %d\n", port);
    return s;
}


int main(int argc, char *argv[]) {
    int port = 9090;
    if (argc > 1) port = atoi(argv[1]);


    towers = calloc(MAX_TOWERS, sizeof(Tower));
    if (!towers) return 1;


    for (int i = 0; i < MAX_TOWERS; i++) towers[i].id = 1001 + i;


    sem_init(&fault_sem, 0, 0);
```

```
    int pipefd[2];
    pipe(pipefd);

    pid_t pid = fork();
    if (pid == 0) {
        close(pipefd[1]);
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[0]);
        recovery_process();
    }

    close(pipefd[0]);
    recovery_fd = pipefd[1];

    int server = start_server(port);
    (void)server;

    pthread_t th[STREAMS];
    int ids[STREAMS];
    for (int i = 0; i < STREAMS; i++) {
        ids[i] = i + 1;
        pthread_create(&th[i], NULL, monitor_stream, &ids[i]);
    }
    for (int i = 0; i < STREAMS; i++) pthread_join(th[i], NULL);

    close(recovery_fd);
    waitpid(pid, NULL, 0);
    close(server);
    sem_destroy(&fault_sem);
    pthread_mutex_destroy(&data_lock);
    pthread_mutex_destroy(&log_lock);
    free(towers);
    printf("[SYSTEM] Monitoring cycle completed successfully.\n");
    return 0;
}
```

9. Sample Output

The following screenshots represent the expected terminal behavior when the program is compiled and executed on a Linux system. They show the control-center server, parallel stream updates, fault alerts and recovery activity occurring while other streams continue processing.

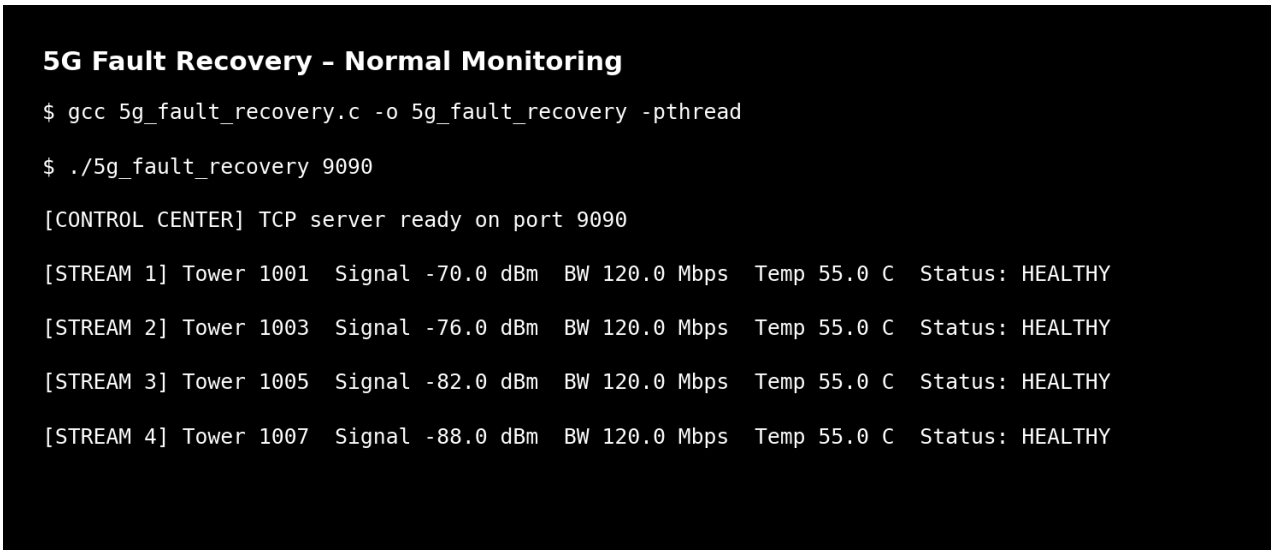

Figure 2. Concurrent processing of healthy tower streams.

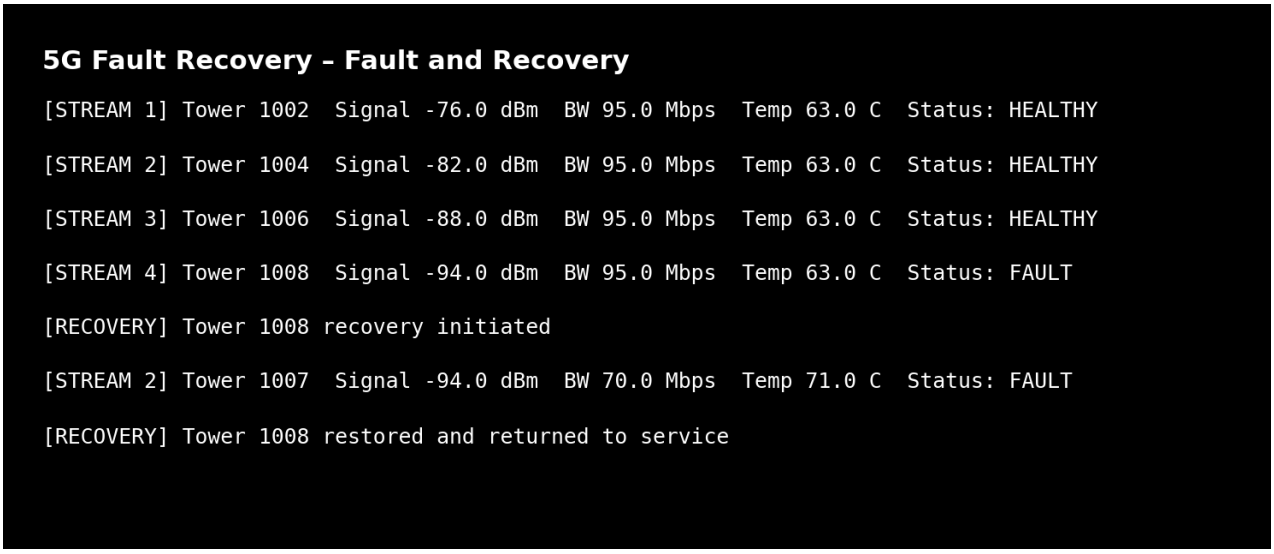

Figure 3. A failed tower is detected and recovered without stopping other streams.

10. Expected Fault Log

fault_history.log

```
Thu Sep 03 18:40:10 2026

Tower 1008: Abnormal telemetry -> recovery requested

Thu Sep 03 18:40:11 2026

Tower 1007: Abnormal telemetry -> recovery requested
```

Figure 4. Example historical fault log generated by the program.

11. Compilation and Execution

1. Save the source code as 5g_fault_recovery.c.
2. Compile using: gcc 5g_fault_recovery.c -o 5g_fault_recovery -pthread
3. Run with the default port: ./5g_fault_recovery
4. Or configure the control-center server port using: ./5g_fault_recovery 9090
5. Inspect fault_history.log after execution to view recorded fault events.

12. Conclusion

The 5G Network Fault Detection and Recovery System demonstrates how C programming and operating-system concepts can be combined to build a practical telecom monitoring application. Dynamic memory allocation provides scalable tower storage, pointers enable direct record updates, and functions separate detection, logging and recovery responsibilities. POSIX threads allow several network streams to be processed concurrently, while mutexes and semaphores provide synchronization. TCP sockets establish the basis for tower-to-control-center communication, and pipe-based IPC separates fault detection from recovery. Most importantly, the architecture supports the hackathon requirement that one tower failure should not block healthy towers. This design can be extended with real sensor inputs, authenticated network messages, persistent databases, automated escalation and more advanced predictive fault analytics for production-scale 5G networks.